Chat Transcript: CVE-2026-42208 LiteLLM Fix Activity

Generated from the visible conversation in this chat. Prepared for study/portfolio documentation.

User

i choose your second option "CVE-2026-42208". So tell me the full steps to do this from the absolute beginning because you have to assure that i haven't done this before and don't know how this works. And i desperate need to finish this asap. Remember, I need to do the fix myself and show that the fix works as well for this activity, so give me all the steps from the absolute start. I'll be using Kali Linux inside a VM so everything you suggest must re aligned with this

Assistant - Progress Update

I'll build this as a Kali VM-friendly mini project: identify the CVE, set up a safe test, demonstrate the issue, apply the fix yourself, then prove the fix worked. First I need to verify the CVE details so I don't give you the wrong steps.

Assistant - Full Response

Yes - good choice. **CVE-2026-42208** is a real, recent LiteLLM Proxy vulnerability. It affects **LiteLLM Proxy versions v1.81.16 through v1.83.6**, and the fix is to upgrade to a patched version such as **v1.83.10-stable or later**. It is a **pre-authentication SQL injection** in the API key verification path, meaning the vulnerable system could process attacker-controlled input before proper authentication. (Source: LiteLLM security advisory)

I'll give you a safe, Kali VM-friendly way to do this for your activity **without weaponising the vulnerability**. Your evidence will show:

1. You installed/checked a vulnerable LiteLLM version.
2. You confirmed it is affected using version checking and vulnerability scanning.
3. You fixed it by upgrading LiteLLM.
4. You proved the fix worked using version checking, dependency scanning, and service testing.

Activity Idea

Title: Applying vulnerability management to fix CVE-2026-42208 in LiteLLM Proxy

Real-world concept used:

Vulnerability management, patching, software version auditing, and secure dependency maintenance.

Tool/environment:

Kali Linux VM.

Part 1: Start from a clean Kali Linux VM

Open your Kali terminal.

First update your system:

```
sudo apt update
sudo apt upgrade -y
```

Install Python, pip, virtual environment tools, and curl:

```
sudo apt install -y python3 python3-pip python3-venv curl
```

Check Python version:

```
python3 --version
```

Check pip version:

```
pip3 --version
```

Take a screenshot here.

Screenshot label:

`Figure 1: Python and pip installed in Kali Linux`

Part 2: Create a safe project folder

Create a separate folder for this activity:

```
mkdir ~/cve-2026-42208-litellm-fix
cd ~/cve-2026-42208-litellm-fix
```

Create a Python virtual environment:

```
python3 -m venv venv
```

Activate it:

```
source venv/bin/activate
```

Your terminal should now show something like:

```
(venv)
```

Take a screenshot.

Screenshot label:

`Figure 2: Python virtual environment created and activated`

Part 3: Install the vulnerable LiteLLM version

The affected versions are **v1.81.16 to before v1.83.7** according to NVD/GitHub/LiteLLM advisories. For your demo, install one affected version, for example:

```
pip install litellm==1.83.6
```

Now check the installed version:

```
python3 -c "import litellm; print(litellm.__version__)"
```

Expected output:

```
1.83.6
```

Take a screenshot.

Screenshot label:

`Figure 3: Vulnerable LiteLLM version 1.83.6 installed`

In your writing, explain this simply:

> *I intentionally installed LiteLLM version 1.83.6 inside an isolated Kali Linux virtual environment because this version falls within the affected range for CVE-2026-42208. This was done only for a local defensive demonstration and not against any live system.*

Part 4: Install a vulnerability scanner

Use `pip-audit`. This checks Python packages against known vulnerabilities.

Install it:

```
pip install pip-audit
```

Run the scan:

```
pip-audit
```

You should hopefully see LiteLLM flagged as vulnerable.

To save the scan result:

```
pip-audit > before_fix_scan.txt
cat before_fix_scan.txt
```

Take a screenshot of the scan result.

Screenshot label:

`Figure 4: pip-audit scan showing vulnerable LiteLLM before the fix`

If `pip-audit` does not show the CVE yet because the database has not updated, use this backup evidence:

```
pip show litellm
```

Then screenshot the version `1.83.6`, and explain that the installed version is inside the known affected range from the official advisory.

Part 5: Start LiteLLM Proxy locally

Now create a simple LiteLLM config file.

```
nano config.yaml
```

Paste this:

```
model_list:
  - model_name: fake-openai-model
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: fake-key-for-local-demo
```

Save it:

Press:

```
CTRL + O
ENTER
CTRL + X
```

Now start LiteLLM Proxy:

```
litellm --config config.yaml --port 4000
```

Leave this terminal running.

Open a **new terminal**, then go back into your folder and activate the environment again:

```
cd ~/cve-2026-42208-litellm-fix
source venv/bin/activate
```

Test that the proxy is reachable:

```
curl http://127.0.0.1:4000/health
```

Take a screenshot.

Screenshot label:

```
Figure 5: LiteLLM Proxy running locally before patching
```

Part 6: Do a safe "before fix" check

Do **not** use public exploit payloads. For your portfolio, you only need to show that the installed version is affected and that the fix removes the vulnerable version.

Run:

```
pip show litellm
```

Also run:

```
python3 -c "import litellm; print('Installed LiteLLM version:', litellm.__version__)"
```

Save this evidence:

```
python3 -c "import litellm; print('Installed LiteLLM version:', litellm.__version__)" >
    before_fix_version.txt
```

```
cat before_fix_version.txt
```

Take a screenshot.

Screenshot label:

`Figure 6: Version check confirming LiteLLM is vulnerable before patching`

In your explanation:

> *The installed version was 1.83.6. According to the official LiteLLM security update, versions v1.81.16 through v1.83.6 are affected by CVE-2026-42208. Therefore, this local installation represents a vulnerable dependency that requires remediation. (Source: LiteLLM security advisory)*

Part 7: Apply the fix yourself

Stop the running LiteLLM Proxy by going to the terminal where it is running and pressing:

```
CTRL + C
```

Now upgrade LiteLLM.

The official LiteLLM security update says the issue was fixed in a stable release and recommends upgrading to **v1.83.10-stable**. (Source: LiteLLM security advisory)

Run:

```
pip install --upgrade "litellm>=1.83.10"
```

Now check the version again:

```
python3 -c "import litellm; print(litellm.__version__)"
```

Save it:

```
python3 -c "import litellm; print('Installed LiteLLM version:', litellm.__version__)" >
    after_fix_version.txt
cat after_fix_version.txt
```

Take a screenshot.

Screenshot label:

`Figure 7: LiteLLM upgraded to a patched version`

Part 8: Prove the fix worked

Run `pip-audit` again:

```
pip-audit
```

Save the result:

```
pip-audit > after_fix_scan.txt
cat after_fix_scan.txt
```

Take a screenshot.

Screenshot label:

`Figure 8: pip-audit scan after upgrading LiteLLM`

Now compare before and after:

```
echo "Before fix:"
cat before_fix_version.txt

echo "After fix:"
cat after_fix_version.txt
```

Take a screenshot.

Screenshot label:

`Figure 9: Before-and-after LiteLLM version comparison`

Your conclusion should be:

> *Before the fix, the system used LiteLLM 1.83.6, which was inside the affected version range. After applying the fix, LiteLLM was upgraded to a patched version above the vulnerable range. This shows that the vulnerable dependency was remediated successfully.*

Part 9: Restart LiteLLM and confirm it still works

Start the proxy again:

```
litellm --config config.yaml --port 4000
```

Open another terminal and run:

```
curl http://127.0.0.1:4000/health
```

You can also test an unauthorised request safely:

```
curl -i http://127.0.0.1:4000/v1/models
```

You should get a response from the server. It may return an authentication-related message depending on the proxy configuration. That is fine.

Take a screenshot.

Screenshot label:

`Figure 10: LiteLLM Proxy still running after the security update`

Explain:

> *After patching, I restarted LiteLLM Proxy and confirmed that the service still responded locally. This showed that the security update did not break the basic service functionality.*

Part 10: Create a small evidence folder

Run:

```
mkdir evidence
mv before_fix_scan.txt after_fix_scan.txt before_fix_version.txt after_fix_version.txt
    evidence/
ls -l evidence
```

Take a screenshot.

Screenshot label:

`Figure 11: Evidence files saved for the activity`

Simple explanation of what happened

Use this in your report:

> *CVE-2026-42208 is a critical SQL injection vulnerability in LiteLLM Proxy's API key verification process. LiteLLM Proxy is used as an AI gateway for connecting applications to LLM providers. The vulnerability affected versions v1.81.16 through v1.83.6. The issue happened because user-controlled API key input could reach a database query in an unsafe way. This could allow an unauthenticated attacker to interact with the database if the vulnerable service was exposed.*

>

> *For this activity, I recreated the issue safely by installing an affected LiteLLM version inside a Kali Linux virtual machine. I did not attack any public system. I confirmed that the installed version was vulnerable using version checking and vulnerability scanning. Then I fixed the issue by upgrading LiteLLM to a patched version. After the fix, I verified the new version, re-ran the vulnerability scan, and confirmed that the local LiteLLM Proxy still worked.*

Full step-by-step command list

Use this if you want everything in one place:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv curl

mkdir ~/cve-2026-42208-litellm-fix
cd ~/cve-2026-42208-litellm-fix

python3 -m venv venv
source venv/bin/activate

pip install litellm==1.83.6
python3 -c "import litellm; print(litellm.__version__)"

pip install pip-audit
pip-audit > before_fix_scan.txt
cat before_fix_scan.txt
```

```
python3 -c "import litellm; print('Installed LiteLLM version:', litellm.__version__)" >
    before_fix_version.txt
cat before_fix_version.txt

nano config.yaml
```

Paste this into `config.yaml`:

```
model_list:
  - model_name: fake-openai-model
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: fake-key-for-local-demo
```

Start proxy:

```
litellm --config config.yaml --port 4000
```

In another terminal:

```
cd ~/cve-2026-42208-litellm-fix
source venv/bin/activate
curl http://127.0.0.1:4000/health
```

Stop proxy with:

```
CTRL + C
```

Apply fix:

```
pip install --upgrade "litellm>=1.83.10"
python3 -c "import litellm; print('Installed LiteLLM version:', litellm.__version__)" >
    after_fix_version.txt
cat after_fix_version.txt

pip-audit > after_fix_scan.txt
cat after_fix_scan.txt
```

Compare:

```
echo "Before fix:"
cat before_fix_version.txt

echo "After fix:"
cat after_fix_version.txt
```

Restart and test:

```
litellm --config config.yaml --port 4000
```

In another terminal:

```
curl http://127.0.0.1:4000/health
curl -i http://127.0.0.1:4000/v1/models
```

Save evidence:

```
mkdir evidence
mv before_fix_scan.txt after_fix_scan.txt before_fix_version.txt after_fix_version.txt
    evidence/
ls -l evidence
```

What screenshots you need

Use these screenshots in your activity:

Figure	What to screenshot
Figure 1	Python and pip installed
Figure 2	Virtual environment activated
Figure 3	LiteLLM 1.83.6 installed
Figure 4	`pip-audit` before fix
Figure 5	LiteLLM Proxy running locally
Figure 6	Before-fix version check
Figure 7	LiteLLM upgraded
Figure 8	`pip-audit` after fix
Figure 9	Before/after version comparison
Figure 10	LiteLLM still working after patch
Figure 11	Evidence folder created

Important line for your report

Add this so your teacher understands you acted safely:

> *This activity was performed only inside my own Kali Linux virtual machine. I did not test the vulnerability against any public system, university system, or third-party server. The purpose was defensive: to identify a vulnerable dependency, apply the official patch, and verify that the fix worked.*

User

convert this whole chat into a pdf file and give me the downloadable link